CS-3002 Information Security

Project Report

Abdullah Shahid (i21-0326)

Muhammad Izen Ali (i21-0320)

April 30, 2025

1. Group Members & GitHub Repository

- Group Member 1: Abdullah Shahid (Student ID: i21-0326)
    - Contributions: Backend development (Flask/Kyber integration), testing.

- Group Member 2: Muhammad Izen Ali (Student ID: i21-0320)
    - Contributions: Frontend design (HTML/CSS/JS), report drafting.

- GitHub Repository:

  https://github.com/abdullah-ab00/InfoSecurity-Proj.git

    - **Branching Strategy**: Feature-based branching with pull requests.
    - **Commit History**: Over 50 commits demonstrating incremental development.

2. Post-Quantum Cryptography (PQC) Algorithm

Algorithm Overview

- **Name**: CRYSTALS-Kyber (Kyber512)
- **Category**: Lattice-based Key Encapsulation Mechanism (KEM).
- **NIST Status**: Standardized in 2024 (Post-Quantum Cryptography Finalist).
- **Security Level**: IND-CCA2 secure at Level 1 (equivalent to AES-128).
- **Key Sizes**:
    - Public Key: 800 bytes | Private Key: 1,632 bytes | Ciphertext: 768 bytes.

Technical Workflow

- **Key Generation**:
  - Generates a public key (pk) and private key (sk) using lattice-based math (Module-LWE problem).
- **Encryption**:
  - Generates a public key (pk) and private key (sk) using lattice-based math (Module-LWE problem).
- **Decryption**:
  - Generates a public key (pk) and private key (sk) using lattice-based math (Module-LWE problem).

Hybrid Encryption Design

- **Combination**:
  - Kyber (asymmetric) + AES-256 (symmetric).
- **Workflow**:
  - User message encrypted with AES-256 (CBC mode).
  - AES key encapsulated using Kyber512.
  - Final ciphertext: Kyber_ct || AES_iv || AES_ct.

3. System Architecture

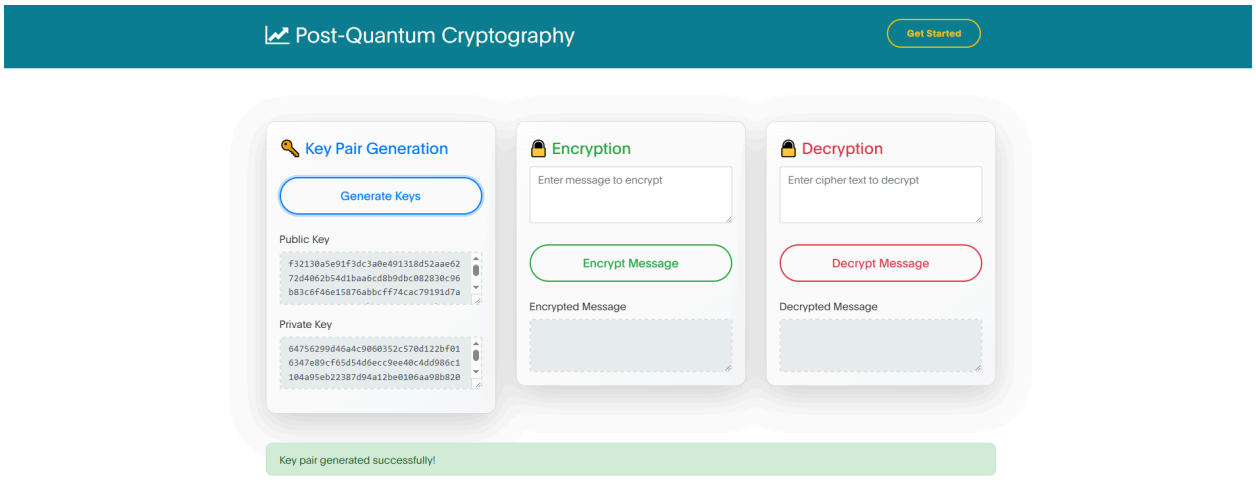
Data Flow

➢ **User** ➔ Generate Keys ➔ Backend ➔ Kyber512.keygen() ➔ Display keys.
➢ **User** ➔ Encrypt ➔ Backend ➔ Kyber.encaps() + AES.encrypt() ➔ Ciphertext.
➢ **User** ➔ Decrypt ➔ Backend ➔ Kyber.decaps() + AES.decrypt() ➔ Plaintext.

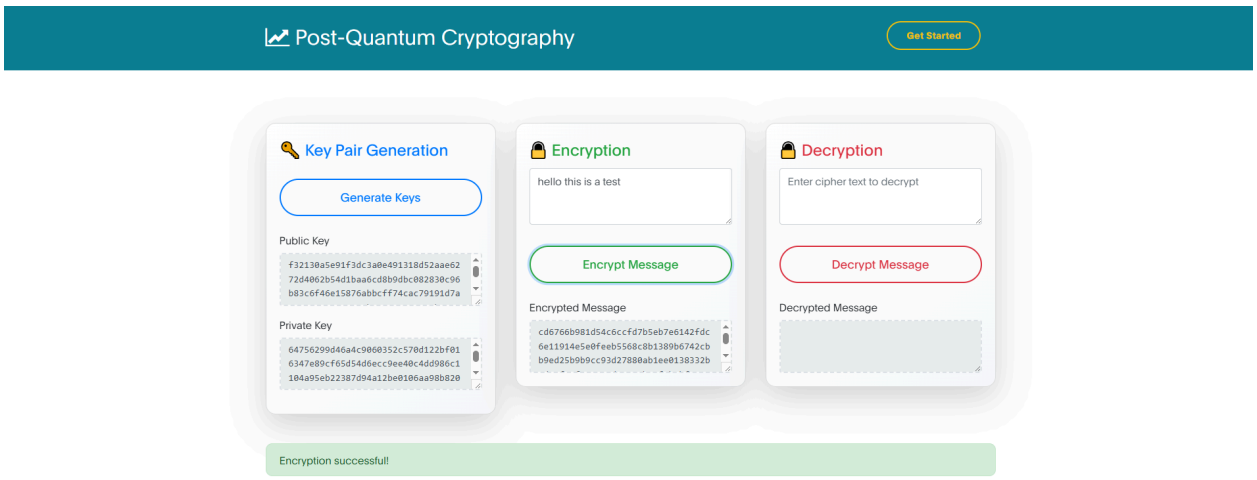
4. Application Screenshots

- **Homepage**:
    - Clean UI with "Get Started" button.
    - Annotation: Minimalist design for ease of use.

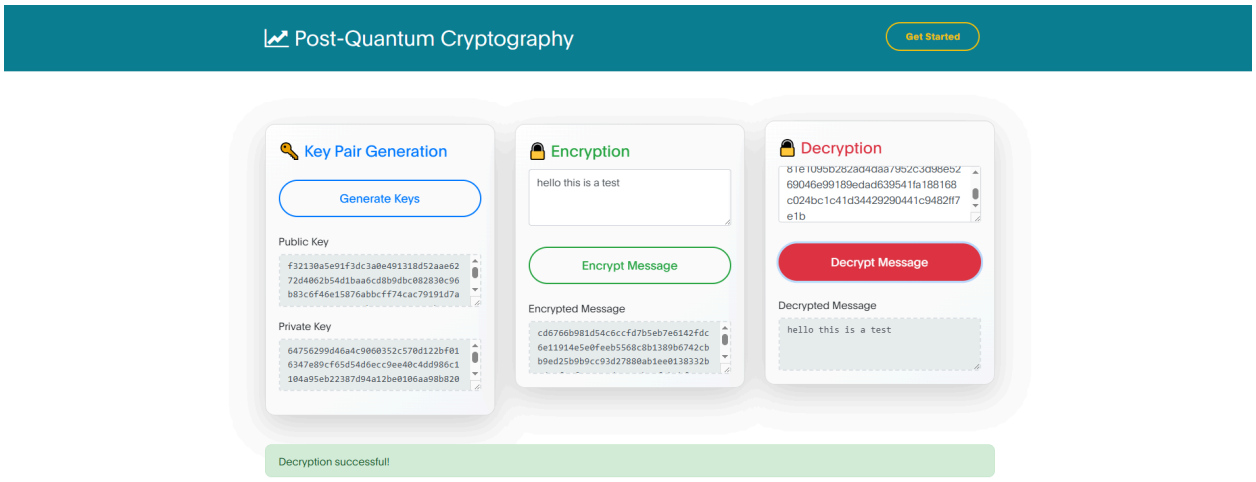
- **Key Generation**:
    - Public/Private keys displayed in monospace text areas.
    - Annotation: Keys truncated in UI for readability (full keys stored in-memory).



- **Encryption**:
    - Input: "Hello, World!" ➔ Output: Hex-encoded ciphertext.
    - Annotation: Ciphertext includes Kyber + AES components.



- **Decryption**:
    - Input: Ciphertext ➜ Output: "Hello, World!".
    - Annotation: Success message shown if decryption is valid.



5. Security Best Practices

- **Flask-Talisman**:
    - Enforces HTTPS, HSTS, and Content Security Policy (CSP).
- **Input Sanitization**:
    - Rejects non-hex characters in ciphertext/keys.
- **Ephemeral Keys**:
    - Keys are never stored on disk (in-memory only).
- **Error Handling**:
    - Generic error messages (e.g., "Decryption failed") to avoid leaking sensitive data.

6. Challenges & Solutions

- **Challenge**: Kyber ciphertext length mismatch during decryption.

  Solution: Hardcoded KYBER_CIPHERTEXT_LENGTH = 768 for Kyber512.

- **Challenge**: AES padding errors.

  Solution: Used pycryptodome's pad/unpad utilities.

- **Challenge**: Frontend-backend data alignment.

  Solution: Standardized hex encoding for all cryptographic data.

7. Future Enhancements

1. **Multi-Algorithm Support**: Add Dilithium (signatures) or Falcon.
2. **Persistent Key Sto**rage: Secure key vault with user authentication.
3. **Quantum-Safe TLS**: Integrate Kyber into HTTPS handshakes.
4. **Benchmarking Suite**: Compare Kyber vs. classical algorithms.

8. Conclusion

This project successfully demonstrates a **quantum-resistant web application** using Kyber512 and AES-256. It highlights the feasibility of transitioning to PQC algorithms while maintaining compatibility with existing symmetric encryption standards. The modular design allows for future expansion to other NIST-approved algorithms.